



Project Overview

- KubeStellar helps manage many clusters from one central place.
- You have a **main cluster** called the core cluster which controls what gets deployed and where. Then you have other clusters called **managed clusters** that actually run the apps.
- Our project fits right into this system.
- We were tasked with building a command-line tool called `kubectl-multi`. It's a plugin that lets you run the same `kubectl` command like `get pods` or `describe services` on multiple clusters at once. So instead of switching contexts over and over, you just run one command and get all the results combined.
- This saves time and makes working across clusters way easier especially for developers.

Goals and Milestones

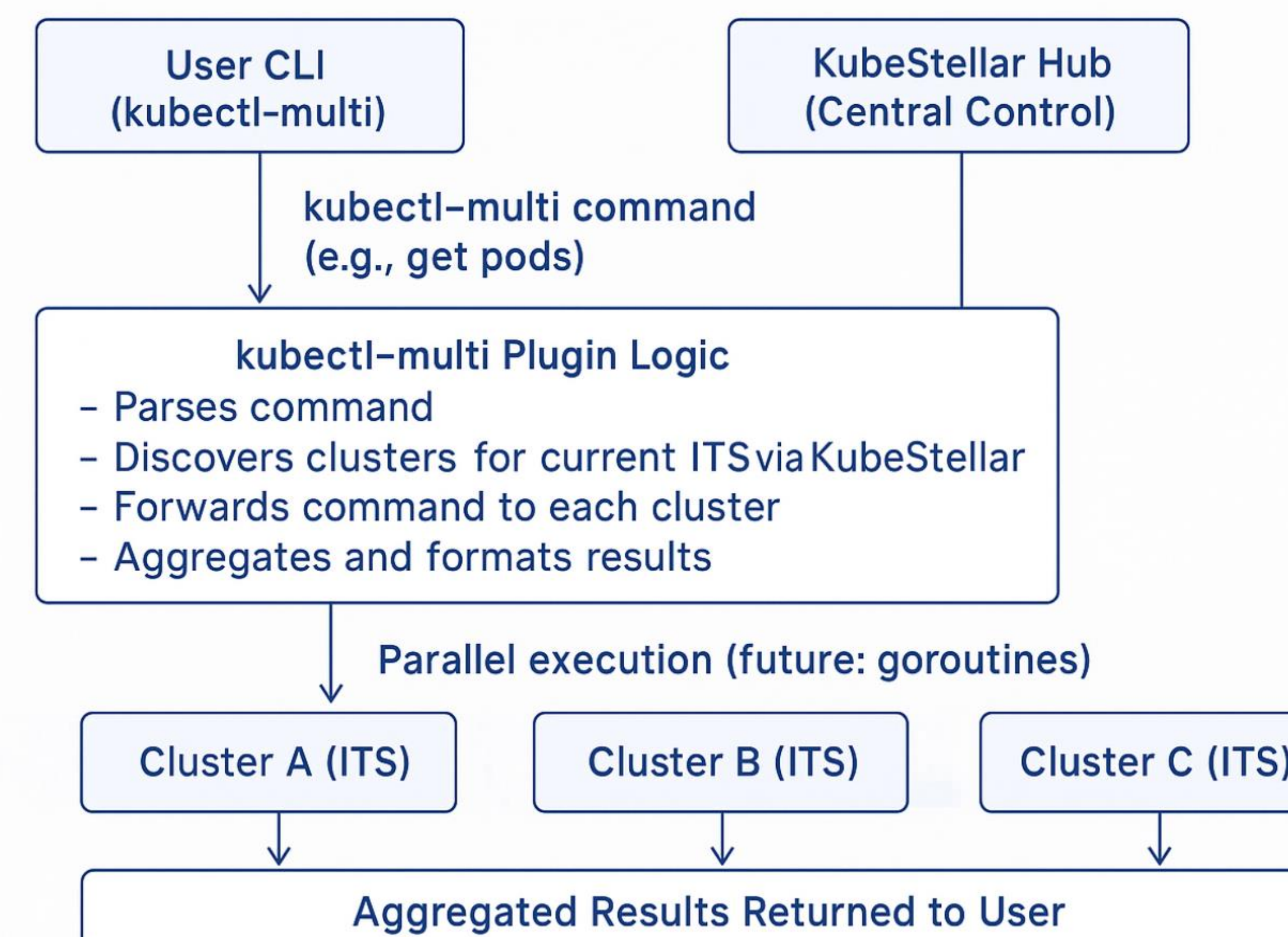
This summer through Georgia Tech's VSIP, we're building a `kubectl-multi` plugin that allows users to run standard `kubectl` commands across multiple Kubernetes contexts. The tool will support operations like `get`, `apply`, and `describe`, and integrate with KubeStellar to surface data from remote clusters. Our first milestone focuses on understanding Kubernetes architecture, context management, and plugin development. The second milestone is to scaffold the plugin in Go using Cobra, implement multi-context command forwarding, and test locally. Eventually, such a plug-in may be integrated into different KubeStellar operations, including the KubeStellar UI, the snapshot command used for debugging in CI/CD pipelines, and other multi-cluster observability and management tasks.

Open Source Outcomes

- Defined GitHub issue templates to streamline collaboration and improve clarity for contributors in a newly defined sub repo of KubeStellar – the multi plug in
- Explored scoped implementations of the plugin to keep contributions focused and manageable
- Collaborated with other CNCF interns, sharing updates and aligning on open source practices
- Learned about Kubernetes as a large-scale open source project under the CNCF, explored the different graduation levels (sandbox, incubating, graduated), and how students can get involved through programs like LFX and GSOC
- Navigated common learning pitfalls in Kubernetes, such as the difference between Kind and Resource

Highlights and Accomplishments – Defining Scope

- The `kubectl-multi` plugin is designed to operate at the ITS (Inventory and Transport Spaces) level, enabling users to run standard `kubectl` commands (such as `get`, `apply`, `describe`, `delete`) across all clusters associated with the current ITS.
- Instead of switching contexts or running commands repeatedly, users can issue a single command to interact with objects across multiple clusters, streamlining multi-cluster management.
- The plugin integrates with KubeStellar, leveraging its architecture to discover clusters and aggregate results, providing a unified view and control plane for distributed Kubernetes environments.



- **ITS Scope:** The plugin operates on the set of clusters defined by the current ITS, ensuring commands are only run where intended.
- **Centralized Control:** KubeStellar provides the mapping between ITS and clusters, enabling dynamic discovery and management.
- **Unified Output:** Results from all clusters are aggregated and presented in a single, coherent output, simplifying multi-cluster operations.
- **Extensible Design:** The architecture supports future enhancements, such as parallel execution and non-blocking operations, to further improve responsiveness and scalability.

Highlights and Accomplishments – Implementing commands

- Reviewed all ~110 `kubectl` commands to identify which ones aligned with our plugin's scope, with a focus on core read operations
- Scaffolded **4–5 key commands including `get`, `apply`, and `logs`** to interact with essential Kubernetes components
- Explored implementation challenges for specific commands and how they behave across different cluster configurations, particularly in multi-namespace ITS setups
- Testing across varied environments proved to be a significant challenge—this remains an ongoing area of work
- Investigated the use of Go's goroutines and channels to enable parallel command execution, improve responsiveness, and avoid blocking from slow clusters -> Ultimately opted for a simpler initial scaffold, but noted this as a strong candidate for future work

Example commands:

- **`kubectl get pods`** - returns a table listing all pods in the current namespace, with columns like NAME, READY, STATUS, RESTARTS, and AGE.
- **`kubectl apply -f`** - Applies a configuration file and returns a success message for each resource created or updated (e.g., "deployment.apps/my-app configured").
- **`kubectl logs <pod>`** - Returns the logs (stdout/stderr) from the specified pod, typically showing application output or error messages.
- **`kubectl describe node`** - Returns detailed information about a node, including capacity, allocatable resources, running pods, taints, and events.
- **`kubectl get services`** - Returns a table of all services in the namespace, with columns like NAME, TYPE, CLUSTER-IP, EXTERNAL-IP, PORT(S), and AGE.

Example of a `get pods` command across multiple clusters `kubectl multi get pods`:

CONTEXT	CLUSTER	NAME	READY	STATUS	RESTARTS	AGE
its1	cluster1	coredns-674b8bbfcf-6k7vc	1/1	Running	2	7d1h
its1	cluster1	coredns-674b8bbfcf-vhh9g	1/1	Running	2	7d1h
its1	cluster1	etcd-cluster1-control-plane	1/1	Running	2	7d1h
its1	cluster1	kindnet-hrpgc	1/1	Running	2	7d1h
its1	cluster1	kube-apiserver-cluster1-control-plane	1/1	Running	2	7d1h
its1	cluster1	kube-controller-manager-cluster1-control-plane	1/1	Running	27	7d1h
its1	cluster1	kube-proxy-tp9qw	1/1	Running	2	7d1h
its1	cluster1	kube-scheduler-cluster1-control-plane	1/1	Running	30	7d1h
its1	cluster2	coredns-674b8bbfcf-5c46s	1/1	Running	2	7d1h
its1	cluster2	coredns-674b8bbfcf-7gft4	1/1	Running	2	7d1h
its1	cluster2	etcd-cluster2-control-plane	1/1	Running	2	7d1h
its1	cluster2	kindnet-nq9zm	1/1	Running	2	7d1h
its1	cluster2	kube-apiserver-cluster2-control-plane	1/1	Running	2	7d1h
its1	cluster2	kube-controller-manager-cluster2-control-plane	1/1	Running	31	7d1h
its1	cluster2	kube-proxy-6fc26	1/1	Running	2	7d1h
its1	cluster2	kube-scheduler-cluster2-control-plane	1/1	Running	29	7d1h
its1	its1-cluster	coredns-68559449b6-g8kpn	1/1	Running	14	7d1h

Future Work

- Implement goroutines and channels for: Parallel Discovery – Process multiple clusters simultaneously and Non-blocking Execution – Avoid delays from slow clusters
- Integration of Multi in core KubeStellar components such as the UI and error logging
- Invite issues and more contributors to our sub repo
- Full finish scaffolding and testing multi!

